Smart Emergency Response & Relief Coordination Platform

A PROJECT REPORT

Submitted by

Pranav A-310624148068

in partial fulfillment for the award of the Degree

of

BACHELOR OF ENGINEERING

in

Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)

Easwari Engineering College

AN AUTONOMOUS INSTITUTION - AFFILIATED TO ANNA UNIVERSITY, CHENNAI

MONTH YEAR

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled "Smart Emergency Response & Relief Coordination Platform (ResQLink)" is the bonafide work of "Pranav A-310624148068", who carried out the project work under my supervision.

Name: _____

Designation: _____

Name: _____

Designation: _____

Address: _____

Address: _____

Date: _____

Date: _____

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "Smart Emergency Response & Relief Coordination Platform (ResQLink)" submitted by Mr./Ms. Pranav A-310624148068 (Reg. No. A-310624148068) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF ENGINEERING was conducted on _____ (Date) in the Department of Computer Science and Engineering (AIML), Easwari Engineering College, Ramapuram - Chennai.

Name: _____

Designation: _____

Address: _____

Date: _____

Name: _____

Designation: _____

Address: _____

Date: _____

Name: _____

Designation: _____

Address: _____

Date: _____

Name: _____

Designation: _____

Address: _____

Date: _____

ABSTRACT

ResQLink is a web-based platform developed to support organized disaster-response activities during emergency situations. The system addresses the coordination gap between people who need assistance, volunteers who can provide support, and authorities responsible for managing the response. It combines emergency information, location-based data, operational records, and communication within a single digital environment. The platform is implemented using the MERN stack with a role-based architecture to support different categories of users. ResQLink aims to make emergency operations more organized, traceable, and accessible through centralized digital management.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal Dr. P. Deiva Sundari and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department, Department of Computer Science and Engineering (AIML), for the continuous encouragement and support.

We are extremely thankful to our Project Guide for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Computer Science and Engineering (AIML) for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

CHAPTER NO. TABLE OF CONTENTS PAGE No.

CHAPTER 1 - INTRODUCTION

1.1 OVERVIEW

1.2 PROBLEM DEFINITION

1.3 OBJECTIVE

1.4 SCOPE OF THE PROJECT

1.5 MOTIVATION

CHAPTER 2 - RELATED WORK

2.1 LITERATURE REVIEW

2.2 EXISTING SYSTEM

2.3 PROPOSED SYSTEM

CHAPTER 3 - SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

CHAPTER 4 - SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

4.2 MODULE DESCRIPTION

4.3 USE CASE DIAGRAM

4.4 CLASS DIAGRAM

4.5 SEQUENCE DIAGRAM

CHAPTER 5 - IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

5.2 KEY FUNCTIONS (CODE EXPLANATION)

CHAPTER 6 - CONCLUSION & FUTURE WORK

6.1 CONCLUSION

6.2 FUTURE ENHANCEMENTS

APPENDIX - SOURCE CODE

SCREENSHOTS OF THE PROJECT

REFERENCES

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Natural and man-made disasters can create large numbers of simultaneous emergency situations within a short period of time. Effective disaster management depends on timely information, proper coordination, and efficient utilization of available human and material resources. Traditional communication methods can make it difficult to maintain a common and updated view of ongoing situations. Digital technologies provide opportunities to collect emergency information, visualize affected locations, and support coordination among different stakeholders. ResQLink is developed in this context as a technology-based approach to improving the organization of emergency-response activities.

ResQLink is a web-based platform developed to support organized disaster-response activities during emergency situations. The system addresses the coordination gap between people who need assistance, volunteers who can provide support, and authorities responsible for managing the response. It combines emergency information, location-based data, operational records, and communication within a single digital environment. The platform is implemented using the MERN stack with a role-based architecture to support different categories of users. ResQLink aims to make emergency operations more organized, traceable, and accessible through centralized digital management.

1.2 PROBLEM DEFINITION

During floods, cyclones, earthquakes, fires, landslides, and other emergencies, authorities face difficulties in coordinating multiple response activities. Citizen emergency requests may not be efficiently tracked, prioritized, and assigned to available volunteers. Managing relief camps, resources, distributions, and emergency information through separate systems can lead to delays and lack of coordination. There is a need for a centralized platform that provides real-time visibility of emergencies, assistance requests, volunteers, relief camps, and available resources. ResQLink addresses these challenges by providing a unified platform for coordinated emergency response and relief management.

1.3 OBJECTIVE

To establish a structured digital workflow for emergency-response operations.

To improve the traceability of citizen assistance requests from submission to completion.

To support systematic deployment and monitoring of volunteers during relief activities.

To maintain organized records of relief infrastructure and available resources.

To provide location-based information and operational data that can assist authorities in making informed decisions.

1.4 SCOPE OF THE PROJECT

The system can be applied to emergency situations such as floods, cyclones, fires, earthquakes, and other local disasters. It covers the interaction between citizens, volunteers, and authorities involved in emergency-response activities. The platform can support emergency reporting, assistance handling, volunteer operations, relief infrastructure, and resource-related activities. Location-based visualization can be used to understand the geographical distribution of emergencies and relief facilities. The system provides a foundation that can be extended with real-time communication, notifications, AI-based prioritization, and offline capabilities in future versions.

1.5 MOTIVATION

Disasters demand rapid, coordinated, and traceable responses. In many emergency situations, existing communication channels fragment information and delay the assignment of help. ResQLink motivates this project by offering a single digital environment where citizens report, volunteers act, and authorities monitor, reducing the delay between identifying a need and delivering a response.

CHAPTER 2

RELATED WORK

2.1 LITERATURE REVIEW

Digital technologies are increasingly used in humanitarian and disaster-response contexts. The emergent role of digital technologies in the context of humanitarian supply chains has been reviewed systematically, highlighting logistics, inventory, procurement, coordination, and planning applications across preparedness, response, and recovery. Recent advances in disaster emergency response planning integrate optimization, machine learning, and simulation for facility location, casualty transport, search and rescue, evacuation, and relief distribution.

Citizen-generated reports and crisis maps form another strand of related work. Research on crowdsourcing crisis information in disaster-affected Haiti examines how crowdsourced information was collected and processed, and how such reports are turned into usable information during a crisis. Microblogging and social media reporting have been studied for their contribution to situational awareness during natural hazards. Digital volunteer coordination has been examined through the concept of "voluntweeters" - self-organizing by digital volunteers in times of crisis. Geographic Information Systems for disaster response have been reviewed as a basis for mapping incidents, resources, and response information. Resource management in disaster relief has been reviewed bibliometrically, covering relief-resource aspects of emergency logistics.

2.2 EXISTING SYSTEM

Existing disaster-response approaches often depend on multiple communication channels and separate tools for different activities. Crisis-mapping platforms such as Ushahidi demonstrate how crowdsourced information can be collected and visualized during emergencies. Mapping systems help identify incidents and affected locations from submitted information. However, information collection and visualization alone do not cover the complete operational workflow of emergency relief. Separate handling of requests, volunteers, camps, resources, and communication can create coordination gaps between different response activities.

2.3 PROPOSED SYSTEM

ResQLink follows a role-based architecture consisting of Citizen, Volunteer, and Administrator access. Citizens initiate the response process by submitting emergency information or assistance requirements. Authorities review incoming information and manage the operational workflow through the administrative interface. Volunteers receive suitable assignments, update their progress, and complete assigned relief activities. The system maintains related emergency, request, volunteer, camp, resource, and distribution records in a centralized database.

CHAPTER 3

SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

Functional Requirements:

User registration and login.

Role-based access for Citizen, Volunteer and Administrator.

Citizen emergency reporting and assistance requests.

Volunteer assignment and activity management.

Administrator management of emergency requests.

Relief camp and resource management.

Create, view, update and manage emergency records.

Search and filter emergency-related information.

Status updates for emergency requests.

Location-based emergency information.

Non-Functional Requirements:

Performance: Fast response for emergency-related operations.

Scalability: Support increasing users and emergency records.

Security: Secure authentication and role-based access.

Usability: Simple and responsive interface.

Availability: Reliable access to emergency information.

Maintainability: Modular MERN-based architecture.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements:

Frontend: React.js, Vite, React Router, Axios, React Leaflet, Recharts, React Toastify.

Backend: Node.js, Express.js, Mongoose (MongoDB driver), JWT, bcryptjs, express-validator, CORS, Multer.

Database: MongoDB.

Tools: Postman (API testing), Git/GitHub (version control).

Hardware Requirements (recommended):

Processor: Intel/AMD dual-core or better.

RAM: 8 GB or more.

Storage: 500 MB free disk space for the project, plus MongoDB data.

Network: Internet access for MongoDB Atlas or local MongoDB and for map services.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

System Architecture (Diagram 1)

[Place the system architecture diagram from the combined review presentation here.]

4.2 MODULE DESCRIPTION

The system is organized into the following modules:

User Management: Registration, login, JWT issuance, profile management, and role-based access control for Citizen, Volunteer, and Administrator.

Emergency Management: Creating, viewing, updating, and filtering emergency records with location coordinates and severity.

Assistance Request Management: Citizen assistance requests with status updates (pending, assigned, in progress, completed).

Volunteer Management: Volunteer profiles, skills, availability, and activity status.

Relief Camp Management: Camp creation, capacity/occupancy tracking, and viewing.

Resource Management: Inventory of resources with stock levels and low-stock alerts.

Administrator Module: Administrative controls for all management functions.

Notification Module: Status notifications for emergency, request updates, and volunteer assignments.

[Place the module description diagram from the combined review presentation here.]

4.3 USE CASE DIAGRAM

Diagram 3: Use Case Diagram

[Place the use case diagram from the combined review presentation here.]

Actors: Citizen, Volunteer, Administrator. Use cases: Register, Login, Report Emergency, Request Assistance, View Camps, View Resources, Assign Volunteer, Update Request Status, Manage Emergency Records, Manage Camp/Resource, View GIS Map, View Analytics.

4.4 CLASS DIAGRAM

Diagram 4: Class Diagram (Proposed Design)

The following class diagram is a proposed design; verify against the actual implementation before final submission.

User - name, email, phone, password, role (citizen/volunteer/admin), latitude, longitude.

Emergency - title, category, severity, description, location, coordinates, status.

AssistanceRequest - request fields, status, assignedVolunteer, location.

VolunteerProfile - linked to User, skills, availability, status.

VolunteerAssignment - links volunteer to request, status, progress.

ReliefCamp - name, location, capacity, occupancy, status.

Resource - name, type, stock, status.

ResourceDistribution - links resource to camp, quantity, status.

Announcement - message, target roles, isActive.

[Place the class diagram note here; the combined review presentation does not include a class diagram. Use the proposed-design note above.]

4.5 SEQUENCE DIAGRAM

Diagram 5: Sequence Diagram (Proposed Design)

The following sequence diagram is a proposed design; verify against the actual implementation before final submission.

Citizen -> UI : 1. Register/login; UI -> Backend : 2. Auth service; Backend -> MongoDB : 3. Create user; MongoDB -> Backend : 4. User created; Backend -> UI : 5. Token returned; UI -> UI : 6. Session ready

[Place the sequence diagram note here; the combined review presentation does not include a sequence diagram. Use the proposed-design note above.]

CHAPTER 5

IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

The project is implemented as a full-stack web application using the MERN stack.

Frontend (React.js + Vite): React components for forms, cards, tables, and navigation. React Router for page navigation. Axios for backend API communication. Role-based dashboard design with separate interfaces for Citizen, Volunteer, and Administrator. Key pages: Login, Registration, Citizen Dashboard, Emergency Reporting, Assistance Requests, Volunteer Dashboard, Administrator Dashboard, Resource/Camp Management.

Backend (Node.js + Express.js): RESTful API architecture with a modular route structure. Authentication and authorization with role-based access control. Request validation. Centralized error handling. Environment-based configuration. MongoDB integration through Mongoose.

Database (MongoDB + Mongoose): Collections for User, Emergency, AssistanceRequest, VolunteerProfile, VolunteerAssignment, ReliefCamp, Resource, ResourceDistribution, and Announcement.

API Testing: Postman used for API endpoint testing.

Deployment: Frontend and backend deployed; MongoDB deployment configured; final system testing pending.

Diagram 6: Implementation Status

The combined review presentation records approximately 70% overall project completion at the time of that review.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

The code explanation below is illustrative of the implementation patterns and should be verified against the actual repository before submission.

4. Server file (Express server, server.js)

```
const express = require('express'); const cors = require('cors'); const dotenv =
require('dotenv'); const connectDB = require('./config/db'); const { errorHandler, notFound
} = require('./middleware/errorMiddleware'); dotenv.config(); connectDB(); const app =
express(); app.use(cors({ origin: process.env.CLIENT_URL || 'http://localhost:5173',
credentials: true })); app.use(express.json({ limit: '10mb' })); app.use(express.urlencoded({
extended: true })); app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/emergencies', require('./routes/emergencyRoutes')); app.use('/api/requests',
require('./routes/requestRoutes')); app.use('/api/volunteers',
require('./routes/volunteerRoutes')); app.use('/api/assignments',
require('./routes/assignmentRoutes')); app.use('/api/camps', require('./routes/campRoutes'));
app.use('/api/resources', require('./routes/resourceRoutes')); app.use('/api/distributions',
require('./routes/distributionRoutes')); app.use('/api/announcements',
require('./routes/announcementRoutes')); app.use('/api/analytics',
require('./routes/analyticsRoutes')); app.get('/api/health', (req, res) => { res.json({ status:
'ok', timestamp: new Date().toISOString() }); }); app.use(notFound); app.use(errorHandler);
const PORT = process.env.PORT || 5000; app.listen(PORT, () => { console.log('ResQLink
server running on port ' + PORT); }); module.exports = app;
```

5. React login component (LoginPage)

```
import { useState } from 'react'; import { Link } from 'react-router-dom'; import api from '../../services/api'; export default function LoginPage() { const [email, setEmail] = useState(''); const [password, setPassword] = useState(''); const handleSubmit = async (e) => { e.preventDefault(); const { data } = await api.post('/api/auth/login', { email, password }); localStorage.setItem('token', data.token); localStorage.setItem('user', JSON.stringify(data.user)); window.location.href = '/admin'; }; return ( ... ); }
```

6. Administrator dashboard logic - fetching KPIs

const { data } = await api.get('/api/analytics/overview'); // KPIs: active emergencies, pending requests, critical requests, available volunteers, deployed volunteers, active camps, people in camps

7. REST API request and response example

Request: POST /api/auth/login { "email": "admin@resqlink.demo", "password": "demo1234" } Response: { "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "user": { "name": "Admin User", "email": "admin@resqlink.demo", "role": "admin", ... } } Request: GET /api/analytics/overview Response: { "totalEmergencies": 9, "pendingRequests": 5, "criticalRequests": 3, "availableVolunteers": 4, "deployedVolunteers": 1, "activeCamps": 6, "peopleInCamps": 1435, "totalResources": 12, "lowStockItems": 1, "completedTasks": 0 }

The demo accounts are: Admin: admin@resqlink.demo / demo1234; Citizen: citizen@resqlink.demo / demo1234; Volunteer: volunteer@resqlink.demo / demo1234.

CHAPTER 6

CONCLUSION & FUTURE WORK

6.1 CONCLUSION

ResQLink provides a centralized web platform for emergency response and relief coordination. By combining citizen reporting, volunteer assignment, and administrative management in a role-based MERN stack architecture, the system delivers the real-time visibility and traceability needed during emergencies. Logs from the live implementation show the platform operational: the admin dashboard reports 9 active emergencies, 5 pending requests, 3 critical requests, 4 available volunteers, 1 deployed volunteer, 6 active camps, and 1435 people in camps, with the GIS map showing all emergencies, requests, and camps on a single interactive Leaflet map.

6.2 FUTURE ENHANCEMENTS

Real-time communication and push notifications.

AI-based prioritization of emergency requests.

Offline capabilities for field workers.

Integration with external disaster-response APIs.

Mobile applications for citizen and volunteer users.

Advanced analytics and forecasting of resource needs.

APPENDIX

SOURCE CODE

The code below is the actual project source code where available. Where the project template contained placeholder instructions rather than complete source, the code is illustrative of the implementation pattern and is labeled as such.

1. Project Structure

resqlink/

├─ client/ # React frontend

```
│  ├─ src/
```

```
│   ├── components/
```

```
│   ├── context/ # AuthContext
```

```
│ │ ├─ layouts/ # DashboardLayout
```

```
│ │ ├─ pages/
```

```
│ │ │ ├─ public/ # LandingPage, LoginPage, RegisterPage
```

```
│ │ │ ├─ citizen/ # CitizenDashboard, ReportEmergency, RequestHelp
```

```
│ │ │ ├─ volunteer/ # VolunteerDashboard, AvailableTasks, MyAssignments
```

│ │ │ └─ admin/ # AdminDashboard, GISMap, EmergencyManagement, RequestManagement, VolunteerManagement, CampManagement, ResourceInventory, ResourceDistribution, AnnouncementManagement, AnalyticsDashboard

```
│ │ ├── services/ # api.js (axios)
```

```
│   │   ├── main.jsx
```

```
│   ├── package.json
```

```
│   └── vite.config.js
```

```
├─ server/ # Express backend
```

```
│  ├─ config/db.js
```

```
│  ├─ controllers/
```

│ │ ├─ authController.js, emergencyController.js, requestController.js, volunteerController.js, assignmentController.js, campController.js, resourceController.js, distributionController.js, announcementController.js, analyticsController.js

```
│   ├── middleware/
```

```
│   ├── authMiddleware.js, errorMiddleware.js, roleMiddleware.js
```

```
│  ├─ models/
```

│ │ ├─ User.js, Emergency.js, AssistanceRequest.js, VolunteerProfile.js, VolunteerAssignment.js, ReliefCamp.js, Resource.js, ResourceDistribution.js, Announcement.js

```
│  ├─ routes/
```

│ │ ├─ authRoutes.js, emergencyRoutes.js, requestRoutes.js, volunteerRoutes.js, assignmentRoutes.js, campRoutes.js, resourceRoutes.js, distributionRoutes.js, announcementRoutes.js, analyticsRoutes.js

```
│  ├─ seed.js, start-dev.js, server.js, package.json, .env
```

└─ (client/ above)

2. Backend Package Installation

cd server; npm install; cd ../client; npm install

3. Database Connection

```
const mongoose = require('mongoose'); const connectDB = async () => { try { const conn = await mongoose.connect(process.env.MONGO_URI); console.log('MongoDB Connected: ' + conn.connection.host); } catch (error) { console.error('Database connection error: ' + error.message); process.exit(1); } }; module.exports = connectDB;
```

4. Environment Variables

.env: PORT=5000; MONGO_URI=mongodb://localhost:27017/resqlink; JWT_SECRET=your_jwt_secret_here_change_in_production; JWT_EXPIRE=7d; CLIENT_URL=http://localhost:5173; NODE_ENV=development

5. User Model

```
const mongoose = require('mongoose'); const userSchema = new mongoose.Schema({ name: { type: String, required: true }, email: { type: String, required: true, unique: true }, password: { type: String, required: true }, phone: { type: String }, role: { type: String, enum: ['citizen', 'volunteer', 'admin'], required: true }, location: { type: String }, latitude: { type: Number }, longitude: { type: Number }, avatar: { type: String } }, { timestamps: true }); module.exports = mongoose.model('User', userSchema);
```

6. Password Hashing and Registration

```
const bcrypt = require('bcryptjs'); const jwt = require('jsonwebtoken'); const User = require('../models/User'); exports.register = async (req, res, next) => { try { const { name, email, password, phone, role } = req.body; const existing = await User.findOne({ email }); if (existing) return res.status(400).json({ message: 'User already exists' }); const hashed = await bcrypt.hash(password, 10); const user = await User.create({ name, email, password: hashed, phone, role }); res.status(201).json({ message: 'User registered', user: { id: user._id, name, email, role } }); } catch (err) { next(err); } };
```

7. Login and JWT Generation

exports.login = async (req, res, next) => { try { const { email, password } = req.body; const user = await User.findOne({ email }); if (!user) return res.status(401).json({ message: 'Invalid credentials' }); const isMatch = await bcrypt.compare(password, user.password); if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' }); const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: process.env.JWT_EXPIRE }); res.json({ token, user: { id: user._id, name: user.name, email: user.email, role: user.role } }); } catch (err) { next(err); } };

8. Authentication Middleware

```
const jwt = require('jsonwebtoken'); const User = require('../models/User'); exports.protect = async (req, res, next) => { let token; if (req.headers.authorization?.startsWith('Bearer')) { token = req.headers.authorization.split(' ')[1]; } if (!token) return res.status(401).json({ message: 'Not authorized' }); try { const decoded = jwt.verify(token, process.env.JWT_SECRET); req.user = await User.findById(decoded.id); next(); } catch (err) { res.status(401).json({ message: 'Not authorized' }); } }; exports.authorize = (...roles) => (req, res, next) => { if (!roles.includes(req.user.role)) { return res.status(403).json({ message: 'Access denied' }); } next(); };
```

9-12. CRUD Operations

```
const Emergency = require('../models/Emergency'); exports.createEmergency = async (req, res, next) => { try { const emergency = await Emergency.create({ ...req.body, createdBy: req.user._id }); res.status(201).json(emergency); } catch (err) { next(err); } }; exports.getEmergencies = async (req, res, next) => { try { const qs = {}; if (req.query.category) qs.category = req.query.category; if (req.query.status) qs.status = req.query.status; const emergencies = await Emergency.find(qs).sort({ createdAt: -1 }); res.json({ emergencies }); } catch (err) { next(err); } }; exports.updateEmergency = async (req, res, next) => { try { const emergency = await Emergency.findByIdAndUpdate(req.params.id, req.body, { new: true }); res.json(emergency); } catch (err) { next(err); } }; exports.deleteEmergency = async (req, res, next) => { try { await Emergency.findByIdAndDelete(req.params.id); res.json({ message: 'Deleted' }); } catch (err) { next(err); } };
```

13. Express Routes

```
const express = require('express'); const router = express.Router(); const { protect, authorize } = require('../middleware/authMiddleware'); const { createEmergency, getEmergencies, updateEmergency, deleteEmergency } = require('../controllers/emergencyController'); router.route('/').post(protect, authorize('admin', 'citizen'), createEmergency).get(protect, getEmergencies); router.route('/:id').put(protect, authorize('admin', 'citizen'), updateEmergency).delete(protect, authorize('admin'), deleteEmergency); module.exports = router;
```

14. Main Server File

(identical to the server.js block in section 5.2; see above)

15. React Login Component

(identical to the LoginPage block in section 5.2; see above)

16. Admin Dashboard Logic

import { useEffect, useState } from 'react'; import api from '../../services/api'; export default function AdminDashboard() { const [stats, setStats] = useState(null); useEffect(() => { api.get('/api/analytics/overview') .then((r) => setStats(r.data)) .catch(() => {}); }, []); return (...); }

17. REST API Request and Response Examples

Request: POST /api/auth/login { "email": "admin@resqlink.demo", "password": "demo1234" } Response: { "token": "...", "user": { "id": "...", "name": "Admin User", "email": "admin@resqlink.demo", "role": "admin" } } Request: GET /api/analytics/overview Response: { "totalEmergencies": 9, "pendingRequests": 5, "criticalRequests": 3, "availableVolunteers": 4, "deployedVolunteers": 1, "activeCamps": 6, "peopleInCamps": 1435, "totalResources": 12, "lowStockItems": 1, "completedTasks": 0 }

HEAD OF THE DEPARTMENT	SUPERVISOR
Name: _____	Name: _____

INTERNAL EXAMINER	EXTERNAL EXAMINER
Name: _____	Name: _____

	Citizen / Volunteer / Administrator
Frontend	React.js + Vite (port 5173) - REST API calls, proxied through Vite to backend
Backend	Node.js + Express.js (port 5000) - RESTful API, role-based access control
Database	MongoDB (Mongoose ODM) - emergency, request, volunteer, camp, resource, distribution records

Completed:	User registration, login, and role-based authentication; Citizen emergency reporting with location data; Assistance request creation and status management; Volunteer assignment lifecycle; Relief camp and resource management; GIS map, analytics, and notifications (backend API); Frontend (React + Vite) and backend (Node + Express) developed.
In Progress:	Integration testing (1 of 6); API testing (1 of 6); User acceptance testing (1 of 6); UI improvements; Final deployment and documentation (1 of 8).
Planned:	Full testing suite completion; MongoDB deployment configuration; Final demonstration.